



Software Engineering Department
Braude College of Engineering
Final Project – Phase A (61998)

Network Stability using a Hyperbolic Geometric latent diffusion model

26-2-R-5

By:

Rotem Maravent

Irit Kogan

Advisors:

Dr. Renata Avros

Prof. Zeev Volkovich

[Link to GitHub](#)

Table of Contents

1.	Introduction	2
2.	Mathematical Background.....	3
2.1.	Mathematical Foundation of Graphs and Latent Space.....	3
2.1.1.	Graph Topology and Matrix Representations	3
2.1.2.	Classical Stability Metrics and their Limitations	3
2.1.3.	Euclidean Limitations and the Anisotropy Problem	4
2.2.	Hyperbolic Geometry in Network Science	4
2.2.1.	Riemannian Manifolds and the Hyperboloid Model	4
2.2.2.	The Popularity-Similarity (PSO) Framework.....	5
2.3.	Denoising Diffusion Probabilistic Models.....	6
2.3.1.	The Forward Diffusion Process in Discrete Time	6
2.3.2.	The Reverse Denoising Process	6
2.3.3.	DDPM Objective Function	6
2.4.	HypDiff Framework and Geometric Measures	7
2.4.1.	Hyperbolic Geometric Autoencoding	7
2.4.2.	Anisotropic Diffusion with Geometric Constraints.....	8
2.4.3.	Hyperbolic Variance and the Fréchet Mean	9
3.	Proposed Methodology.....	10
3.1.	System Overview and Architecture.....	10
3.2.	Graph Perturbation Module.....	10
3.3.	Generative Recovery Module	11
3.4.	Probabilistic Stability Scoring Module	12
4.	Evaluation and Verification Plan	12
4.1.	Success Metrics and Criteria	13
4.2.	Evaluation Datasets.....	14
4.2.1.	Synthetic Datasets	14
4.2.2.	Real-World Datasets	14
4.3.	Planned Experiments and Expected Results	16
4.4.	Formal Software Testing Strategy.....	17
5.	System Infrastructure and Project Roadmap.....	18
5.1.	System Requirements.....	18
5.2.	Expected Challenges	19
5.3.	Future Work Plan and Project Roadmap	19
5.4.	Ai Tools Usage.....	20

Abstract

Evaluating complex network resilience is critical across critical infrastructures, biological systems, and social structures. However, classical graph-theoretic metrics assess networks as deterministic, fixed topologies, completely ignoring structural uncertainty under dynamic stress. While deep learning alternatives like Graph Neural Networks (GNNs) attempt to model these graphs, they introduce severe geometric distortion by forcing non-Euclidean structures into flat Euclidean spaces, while standard isotropic diffusion models destroy delicate topological information. To address these limitations, this project proposes a novel network stability evaluation framework by adapting the state-of-the-art Hyperbolic Geometric Latent Diffusion Model (HypDiff). By leveraging continuous hyperbolic geometry, the framework naturally captures hierarchical scale-free topologies without distortion. The proposed architecture systematically applies controlled edge disruptions, maps the damaged graphs into a hyperbolic latent space, and utilizes an anisotropic reverse denoising process to probabilistically reconstruct missing links. Finally, a dedicated scoring module aggregates recovery frequencies to calculate a dynamic, uncertainty-aware stability score ($S_{stability}$). The framework will be validated on synthetic models (SBM, BA) and real-world benchmarks (IMDB-B, PROTEINS) to provide a precise, geometry-derived tool for quantifying structural robustness.

1. Introduction

Complex networks appear in many real-world domains, spanning from essential infrastructures such as power grids and communication lines to biological maps of protein interactions and social networks. In practice, these topologies are rarely static and remain constantly exposed to environmental stress, dynamic loads, or targeted structural failures (Artime et al., 2024). Classical approaches to evaluating the resilience and stability of these complex systems have relied on classical graph theoretic metrics like algebraic connectivity, betweenness centrality and effective resistance (Artime et al., 2024; Ellens & Kooij, 2013). Although these traditional tools are informative, they suffer from a severe conceptual flaw by treating each network as a single, fixed point in a continuous representation space. This deterministic assumption completely ignores structural uncertainty, failing to account for the multiple probabilistic states a network can occupy under stress. Consequently, a stability evaluation tool incapable of representing structural uncertainty cannot provide a reliable assessment of true system resilience.

Attempting to capture this intricate topology using modern deep learning approaches, such as Graph Neural Networks, presents a critical bottleneck. Traditionally, these models force complex, non-Euclidean graph structures into flat Euclidean latent spaces, which inherently cause massive geometric distortion (Chami et al., 2019). Furthermore, applying standard generative diffusion models to these distorted embeddings ultimately destroys the network's delicate structural information, fundamentally preventing accurate measurements of true stability under perturbation (Fu et al., 2024; Yang et al., 2023).

To address these geometric challenges, recent research demonstrates that hyperbolic space naturally models hierarchical structures without distortion. Concurrently, generative diffusion models, such as GDSS and DiGress, have proven highly effective for learning probabilistic graph representations by treating graphs as distributions rather than single points (Jo et al., 2022; Vignac et al., 2022). Combining these two paradigms, the state-of-the-art HypDiff model (Fu et al., 2024) established a latent diffusion framework operating within hyperbolic space. However,

HypDiff was engineered strictly for graph generation. Applying such probabilistic models to measure network stability remains an unresolved research direction. Specifically, connecting continuous geometry to structural resilience is a major open challenge (Arttime et al., 2024). This project bridges this gap by adapting the HypDiff framework to measure stability, representing networks as distributions over potential structural states.

This project proposes a network stability evaluation framework based on a Hyperbolic Geometric Latent Diffusion Model. The architecture applies controlled edge disruptions, maps the damaged graph into a hyperbolic latent space, and utilizes an anisotropic reverse denoising process to probabilistically reconstruct missing links. Finally, a scoring module calculates a dynamic stability score based on the recovery frequency of removed edges, providing a precise, geometry derived tool to quantify structural robustness.

2. Mathematical Background

2.1. Mathematical Foundation of Graphs and Latent Space

2.1.1. Graph Topology and Matrix Representations

A complex network is mathematically abstracted as a graph $\mathcal{G} = (V, E)$, where V is the set of nodes and E is the set of edges representing their interactions (Ellens & Kooij, 2013). The network's topology is represented by an $n \times n$ adjacency matrix A , where $A_{ij} = 1$ if an edge connects nodes i and j , and $A_{ij} = 0$ otherwise.

Beyond its topology, each node i is additionally associated with a d -dimensional feature vector x_i describing its individual attributes. These vectors form the $n \times d$ node feature matrix X , such that the complete input representation processed by geometric learning frameworks is given by $G = \{X, A\}$ (Fu et al., 2024). In datasets lacking explicit node attributes, X is conventionally initialized from structural properties or as an identity matrix, ensuring that the representation remains well-defined across all benchmarks.

Node connectivity is quantified by its degree k_i , representing its total number of links (Arttime et al., 2024). These degrees form the diagonal degree matrix D , where $D_{ii} = k_i$.

Finally, the graph Laplacian L , defined as $L = D - A$ (Ellens & Kooij, 2013), is a crucial matrix representation. Its spectral properties are foundational in geometric machine learning for defining structural filters and formulating continuous-time diffusion processes (Elhag et al., 2022).

2.1.2. Classical Stability Metrics and their Limitations

Traditionally, network robustness is evaluated using deterministic topological metrics that quantify a network's vulnerability to failures or attacks (Ellens & Kooij, 2013). Common spectral and spatial measures include Algebraic Connectivity (the second smallest eigenvalue of the Laplacian matrix, λ_2), which reflects how easily a graph can be disconnected, and Betweenness Centrality, which identifies structural bottlenecks based on the fraction of shortest paths passing through a specific node or edge (Ellens & Kooij, 2013).

Despite their utility in classical network science, these metrics suffer from a fundamental limitation: they evaluate networks as deterministic, fixed topologies. By assuming the network

is a fixed point, classical metrics are unable to capture structural uncertainty or represent the probabilistic nature of networks operating under dynamic stress (Artime et al., 2024). This deterministic bottleneck necessitates a shift toward probabilistic, generative frameworks to properly evaluate true network stability.

2.1.3. Euclidean Limitations and the Anisotropy Problem

Graph Neural Networks (GNNs) traditionally embed nodes into Euclidean space. However, real world networks frequently exhibit hierarchical and scale free topologies where the graph volume grows exponentially. Embedding these complex structures into a Euclidean space, which only grows polynomially with respect to the radius, inherently incurs massive geometric distortion (Chami et al., 2019).

Furthermore, graph representations in Euclidean latent space suffer from the anisotropy problem. Unlike regular grid like data (such as images) that distribute isotropically around the origin, non-Euclidean graph embeddings are highly anisotropic, concentrating heavily along specific directional axes (Fu et al., 2024). Consequently, applying standard continuous time diffusion models which rely on injecting isotropic Gaussian noise is fundamentally flawed for graphs.

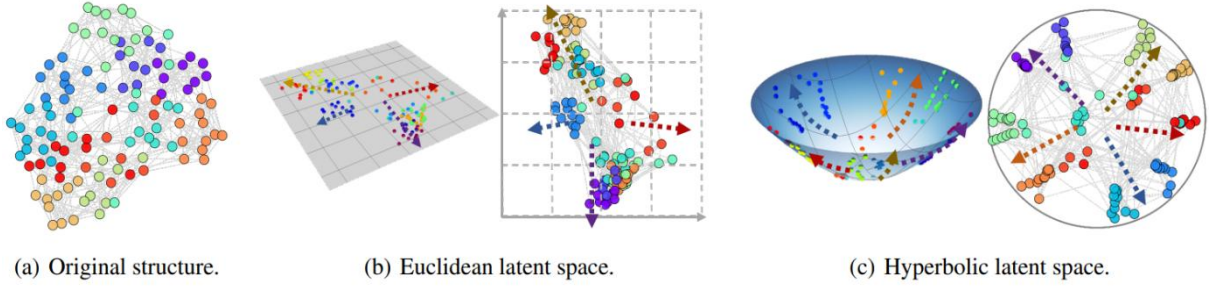


Figure 1. Visualization of NCAA football graph embeddings via SVD demonstrating the anisotropy problem. The figure shows the original structure (a) compared against its 2D Euclidean projection (b) and 2D hyperbolic Poincaré disk projection (c) (Fu et al., 2024).

Isotropic noise cannot distinguish the anisotropic structural signal from random noise, causing the signal to noise ratio (SNR) to decline rapidly towards zero (Yang et al., 2023). Ultimately, this isotropic diffusion treats the anisotropic structural information as noise, destroying the underlying topological information upon which true network stability analysis depends (Fu et al., 2024; Yang et al., 2023).

2.2. Hyperbolic Geometry in Network Science

2.2.1. Riemannian Manifolds and the Hyperboloid Model

Hyperbolic geometry is a non-Euclidean geometry characterized by a constant negative curvature, making it naturally equipped to model continuous hierarchical structures (Nickel & Kiela, 2017). Among its isometric models, the hyperboloid model (also known as the Lorentz model) is highly preferred in graph machine learning due to its rigorous numerical stability (Chami et al., 2019). Given two points x and y , the Minkowski inner product is mathematically defined as:

$$\langle x, y \rangle_{\mathcal{L}} = -x_0 y_0 + \sum_{i=1}^d x_i y_i,$$

the d dimensional hyperboloid manifold $\mathbb{H}^{d,K}$ with constant negative curvature $-1/K$ (where $K > 0$) is mathematically defined as the set of points $x \in \mathbb{R}^{d+1}$ satisfying the conditions:

$$\langle x, x \rangle_{\mathcal{L}} = -K, \quad x_0 > 0.$$

Since hyperbolic space lacks a standard vector space structure, geometric operations are fundamentally performed in the Euclidean tangent space $\mathcal{T}_x \mathbb{H}^{d,K}$, which provides a local approximation of the manifold at point x . The mapping between the hyperbolic manifold and this Euclidean tangent space is achieved through two bijective operations: the logarithmic map $\text{logmap}_x^K(y)$ projects a point y from the manifold onto the tangent space, while the exponential map $\text{expmap}_x^K(v)$ projects a tangent vector v back onto the hyperboloid manifold (Chami et al., 2019). These maps establish the required mathematical foundation to perform diffusion processes directly within the hyperbolic space.

2.2.2. The Popularity-Similarity (PSO) Framework

The Popularity Similarity optimization framework provides a geometric interpretation of complex network topologies by mapping nodes into a polar coordinate system within the hyperbolic space. In this latent space, the radial coordinate r represents the popularity of a node. Nodes located closer to the origin possess higher popularity and consequently attract more connections. Conversely, the angular coordinate θ abstracts the structural similarity between nodes, dictating local clustering and community formations (Papadopoulos et al., 2012).

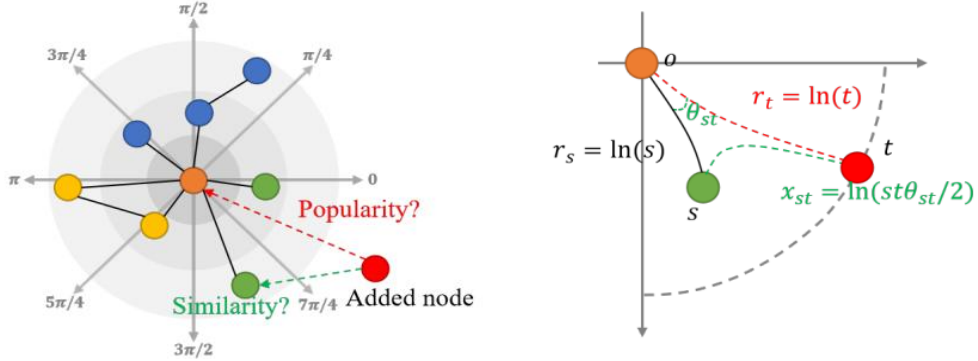


Figure 2. Illustration of the Popularity-Similarity Optimization (PSO) framework within hyperbolic space. The diagram demonstrates how a node's radial coordinate corresponds to its popularity, while the angular coordinate reflects structural similarity (Fu et al., 2024).

The hyperbolic distance between two nodes s and t , possessing polar coordinates (r_s, θ_s) and (r_t, θ_t) , acts as a unified metric combining both attractiveness attributes. This distance is mathematically approximated as:

$$x_{st} = r_s + r_t + \ln\left(\frac{\theta_{st}}{2}\right),$$

where θ_{st} represents the angular distance between the two nodes (Papadopoulos et al., 2012). The framework demonstrates that complex networks naturally grow by minimizing this distance, thereby establishing connections that optimally balance global popularity and local similarity.

2.3. Denoising Diffusion Probabilistic Models

2.3.1. The Forward Diffusion Process in Discrete Time

Denoising Diffusion Probabilistic Models operate by gradually perturbing an initial data distribution with Gaussian noise over a sequence of discrete time steps $t \in \{1, \dots, T\}$ (Ho et al., 2020). Given an initial data representation x_0 , the forward diffusion process is mathematically defined as a Markov chain that progressively injects noise according to a predefined variance schedule β_t . The transition probability at each discrete step is formulated as:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I),$$

where $\mathcal{N}$ denotes the normal distribution and I represents the identity matrix. By defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, the Markovian property allows for direct sampling of the noisy state at any arbitrary time step t using the closed form equation:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

As the time step t approaches the maximum limit T , the original structural signal is completely transformed into an isotropic Gaussian noise distribution, providing a tractable prior for the generative reconstruction (Huang et al., 2022).

2.3.2. The Reverse Denoising Process

To generate new data, the model must reverse the forward perturbation by gradually removing the noise. Since the exact reverse transition probability is computationally intractable, Denoising Diffusion Probabilistic Models employ a neural network with learnable parameters θ to approximate this reverse time Markov chain (Ho et al., 2020). The approximated reverse process is mathematically defined as:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)),$$

where μ_θ and Σ_θ represent the predicted mean and covariance matrices computed by the network at a specific time step t . In practical implementations, especially those adapted for complex topological structures like graphs, the denoising network often directly predicts the original clean data x_0 instead of estimating the intermediate noise parameters (Yang et al., 2023). By recursively applying this learned reverse transition starting from pure isotropic Gaussian noise at the maximum step T , the model progressively reconstructs the geometric structure and generates high fidelity samples out of the initial noise distribution.

2.3.3. DDPM Objective Function

To optimize the neural network parameters θ , Denoising Diffusion Probabilistic Models traditionally minimize a variational bound on the negative log likelihood of the data (Ho et al., 2020). This optimization is mathematically equivalent to a denoising score matching objective, which measures the mean squared error between the true added noise and the predicted noise.

However, for complex non-Euclidean structures, recent graph diffusion frameworks directly predict the clean initial state rather than estimating the intermediate noise (Yang et al., 2023). Consequently, the objective function is formulated to minimize the expected Euclidean distance

between the original unperturbed feature representation X_0 and the predicted feature representation computed by the denoising network f_θ . Given the noisy features X_t at a uniformly sampled time step t and the adjacency matrix A , the dedicated loss function $\mathcal{L}_{HDM}$ (Fu et al., 2024) is formally defined as:

$$\mathcal{L}_{HDM} = \mathbb{E}_{X_0, t} [\| f_\theta(X_t, A, t) - X_0 \|^2].$$

By optimizing this objective, the network effectively learns to map the isotropic noise distribution back to the original topological structure, establishing a stable generative engine (Fu et al., 2024).

2.4. HypDiff Framework and Geometric Measures

2.4.1. Hyperbolic Geometric Autoencoding

To process discrete graph structures within a continuous domain, the HypDiff framework employs a hyperbolic geometric autoencoder (Fu et al., 2024). This architecture consists of an encoder that embeds the graph into a hyperbolic manifold and a Fermi-Dirac decoder that reconstructs the graph topology. The encoder is implemented as a Hyperbolic Graph Convolutional Neural Network, which utilizes exponential and logarithmic maps to project features between the hyperbolic manifold and its corresponding Euclidean tangent spaces (Chami et al., 2019).

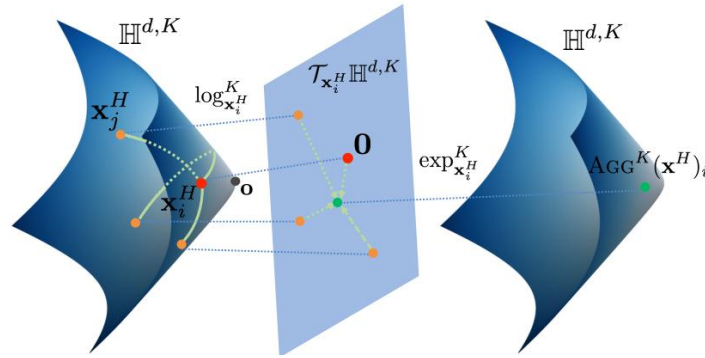


Figure 3. The neighborhood aggregation process in HGCN. Node embeddings are first projected onto the Euclidean tangent space for aggregation via the logarithmic map and subsequently mapped back to the hyperbolic manifold using the exponential map (Chami et al., 2019).

After the graph is embedded and processed within the latent space, the Fermi-Dirac Decoder algorithm calculates the precise probability of edge connections between nodes (Fu et al., 2024; Nickel & Kiela, 2017). It computes these probabilities by converting their continuous hyperbolic distances (utilizing the metric x_{st} defined previously in the PSO framework) into discrete edge probabilities. Mathematically, the probability of an edge existing between node i and node j is computed using the Fermi-Dirac distribution:

$$P(A_{i,j} = 1) = \frac{1}{e^{(x_{st}-r)/t} + 1},$$

where r and t are hyperparameters controlling the scale and steepness of the activation. This algorithmic step explicitly bridges the continuous latent state and the discrete graph topology to

reconstruct the edges. To optimize this embedding process, the framework minimizes a specific reconstruction loss function for the autoencoder. Given the original graph data x and its latent hyperbolic representation z_x , the autoencoder loss $\mathcal{L}_{HAE}$ is mathematically defined as:

$$\mathcal{L}_{HAE} = -\mathbb{E}_{q_\phi(z_x|x)}[\logmap_o^c p_\xi(x | z_x)],$$

in this equation q_ϕ represents the approximate posterior distribution produced by the hyperbolic encoder with learnable parameters ϕ and p_ξ denotes the likelihood distribution modeled by the Fermi-Dirac decoder with learnable parameters ξ (Fu et al., 2024). Furthermore, $\logmap_o^c$ refers to the logarithmic mapping to the north pole origin o of the hyperbolic space. According to Fu et al. (2024), using this specific mapping simplifies the computation by allowing the model to approximate the hyperbolic Gaussian distribution using the Gaussian distribution of the tangent plane.

2.4.2. Anisotropic Diffusion with Geometric Constraints

Standard diffusion models inject isotropic white noise, which rapidly degrades the signal to noise ratio when applied to highly anisotropic non-Euclidean graph embeddings (Yang et al., 2023). To preserve the underlying topology, the framework introduces a directional diffusion approach guided by specific geometric boundaries (Fu et al., 2024).

Instead of uniform noise, the model utilizes an angular similarity constraint to preserve local community structures during the diffusion process (Fu et al., 2024). To accurately apply these constraints, the network nodes must first be grouped into structural communities. As proposed by Fu et al. (2024), the system achieves this during a preprocessing stage by utilizing the Hyperbolic-Kmeans algorithm to cluster the nodes based on their geometric similarity. This clustering process identifies the hyperbolic center for each community, denoted as $h_{\mu i}$ for a given node i . The angular constrained Gaussian noise z is then calculated relative to this specific community center as follows:

$$z = \text{sgn}(\logmap_o^c(h_{\mu i})) * \epsilon,$$

where $\logmap_o^c$ is the logarithmic map at the origin o , sgn is the sign function, and ϵ represents standard Gaussian noise (Fu et al., 2024).

This directional noise z ensures that the perturbation aligns with the topological clustering of the network. Combined with a radial popularity constraint that regulates the diffusion strength along the radial axis, this anisotropic approach prevents the structural information from being treated as random noise, thereby maintaining the scale free properties of the original graph throughout the latent diffusion process (Fu et al., 2024).

To fully capture the network structural properties during the forward process, the HypDiff model proposed by Fu et al. (2024) integrates both the angular similarity constraints and the radial popularity constraints into a single unified diffusion equation. By combining the angular constrained noise z with a radial growth control mechanism, the complete geometric forward diffusion process is defined as:

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}z + \delta \tanh\left[\frac{\sqrt{c}\lambda_c^o t}{T_0}\right]x_0,$$

in this final equation, z represents the angular constrained Gaussian noise that preserves community structures, while the trigonometric term acts as the radial constraint. According to Fu et al. (2024), the parameter δ is the radial popularity coefficient, which explicitly controls the diffusion strength of each node in the hyperbolic space. This specific radial constraint ensures that the inherent popularity of central nodes is maintained throughout the diffusion steps, preventing the structural degradation typically caused by standard isotropic noise.

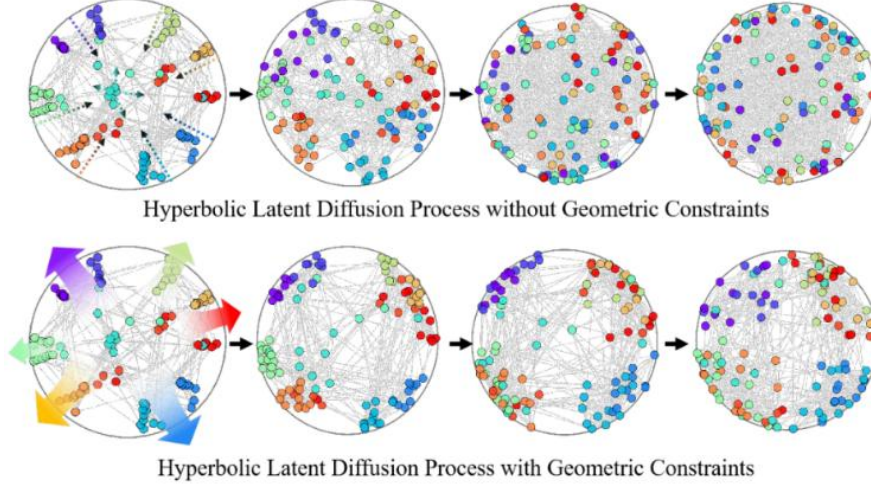


Figure 4. Hyperbolic latent diffusion processing with isotropic/anisotropic noise (Fu et al., 2024).

2.4.3. Hyperbolic Variance and the Fréchet Mean

In continuous hyperbolic spaces, defining standard statistical moments such as mean and variance is mathematically complex due to the absence of a vector space structure. The natural geometric equivalent of the Euclidean average is the Fréchet mean (Chami et al., 2019). However, calculating the exact Fréchet mean is computationally intractable because it has no closed form solution. To compute variance and aggregate features efficiently, frameworks must leverage the Euclidean properties of the tangent space (Fu et al., 2024). Instead of solving the Fréchet mean directly on the manifold, the model projects the hyperbolic embeddings into the tangent space of a specific reference point using the logarithmic map. Once in this Euclidean domain, standard statistical operations such as attention-based neighborhood aggregation can be performed safely (Chami et al., 2019). Finally, the aggregated Euclidean results are projected back onto the hyperbolic manifold using the exponential map. By computing the variance and average locally in the tangent space, the model effectively approximates the Fréchet mean, ensuring stable optimization without distorting the geometric topology (Fu et al., 2024).

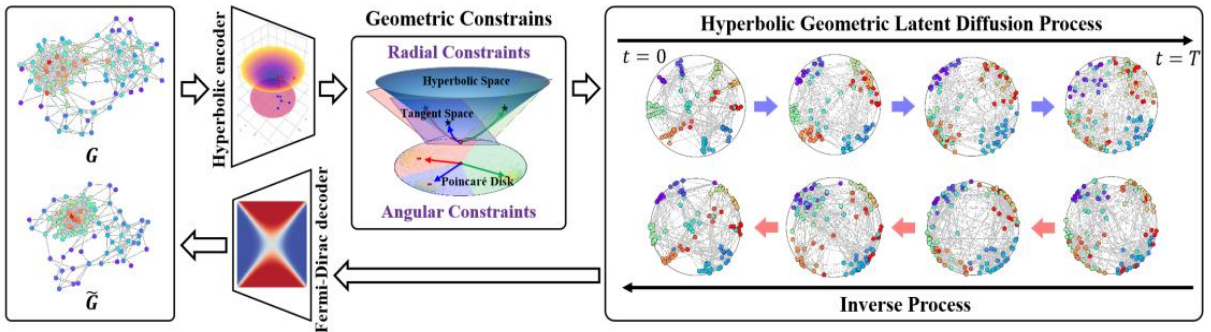


Figure 5. An illustration of the HypDiff architecture detailing the hyperbolic geometric autoencoder and the latent diffusion process (Fu et al., 2024).

3. Proposed Methodology

3.1. System Overview and Architecture

This chapter details the engineering pipeline for evaluating network stability using the HypDiff framework. The complete methodology is illustrated in the system architecture diagram below.

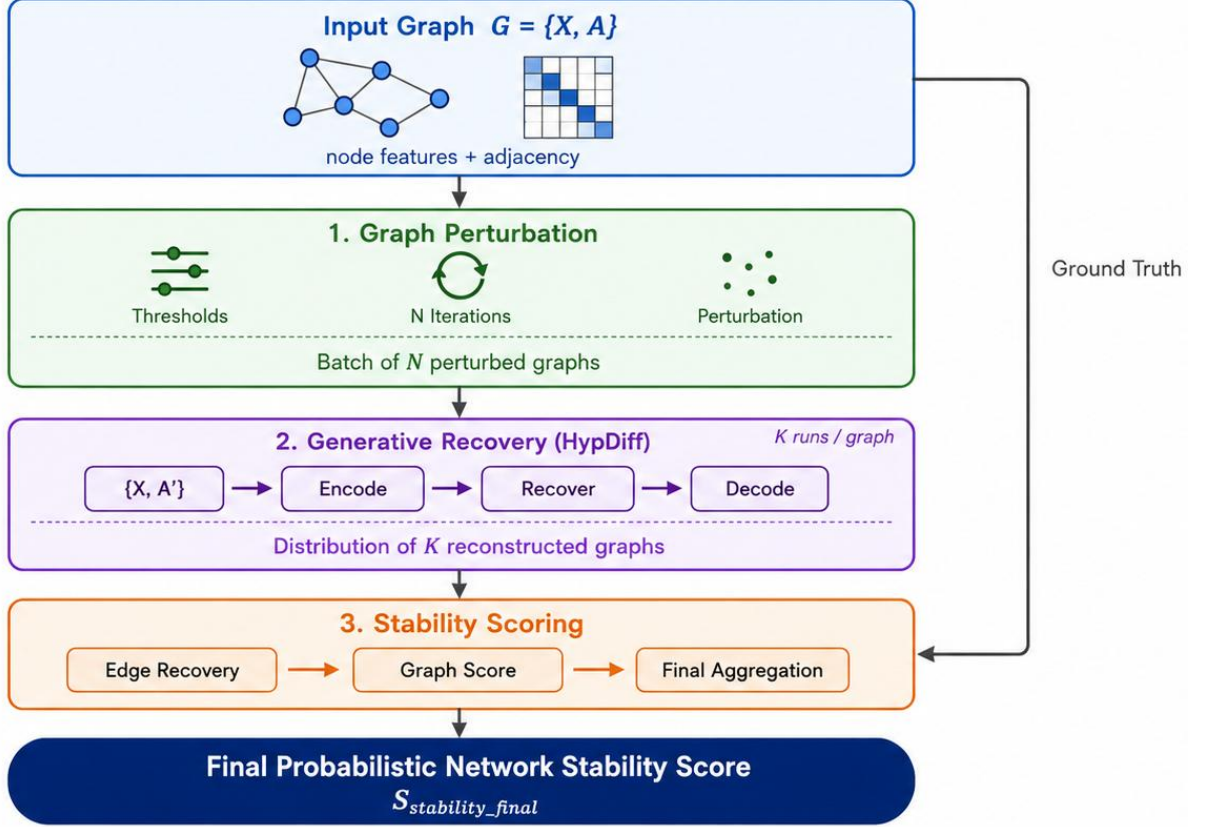


Figure 6. The proposed HypDiff pipeline architecture for network stability analysis.

As depicted in the architecture diagram, the proposed system operates through three primary sequential modules supported by a ground truth bypass connection for the final evaluation. First, the Graph Perturbation Module subjects the raw input graph to targeted attacks or random edge removals, repeating this process for N independent iterations to output a batch of N perturbed graphs. Each perturbed graph serves as the direct input for the Generative Recovery Module, where it is processed through the pre-trained HypDiff model. Within this module, the graph is mapped into a continuous hyperbolic latent space representation (z_x) using a Hyperbolic Geometric Autoencoder, modified with partial forward noise only up to a truncation step $t^* < T$, and restored via an anisotropic reverse denoising process and a Fermi-Dirac decoder. Executed K independent times per perturbed graph, this process outputs a diverse distribution of reconstructed graphs. Finally, the Probabilistic Stability Scoring Module receives this generated distribution and compares it directly against the raw input graph, which bypasses the generative model to act as the absolute structural ground truth. Through the sequential stages of edge recovery tracking, graph-level scoring, and final aggregation, this final evaluation block calculates how frequently the removed edges reappear, outputting the final probabilistic network stability score. This entire pipeline is executed once per damage threshold (e.g., 10% to 50% edge removal), tracing how stability degrades as damage increases.

3.2. Graph Perturbation Module

The Graph Perturbation Module serves as the initial active phase of the pipeline. It receives the raw input graph, mathematically defined as $G = \{X, A\}$. All perturbations are applied exclusively to the adjacency matrix A , while the node set V and the node feature matrix X propagate through the pipeline unchanged. To systematically evaluate the network’s resilience, the module processes the input graph through two distinct and parameterized deletion methods. To comprehensively map the structural degradation, these removals are evaluated across a spectrum of incrementally increasing percentage thresholds (e.g., 10%, 20%, 30%), simulating increasingly aggressive topological disruptions:

- **Random Edge Removals:** Algorithmically, the system iterates over the edge set E and removes edges based on a uniform probability distribution. The primary parameter controlling this process is the random removal fraction, denoted as P_{rand} . The system utilizes P_{rand} to incrementally remove specific percentages of the total edges, outputting the randomly perturbed adjacency matrix A' .
- **Targeted Attacks:** In this method, the system evaluates and ranks edges based on the Degree Centrality of the two nodes they connect. The degree of a node, denoted as k_i , simply quantifies the total number of links involving that specific node. Specifically, for any given edge e connecting node i and node j , the system calculates an importance score based on their combined degrees, defined mathematically as $Score(e) = k_i \times k_j$. The controlling parameter for this process is the targeted removal fraction, denoted as P_{target} . Once all edges are scored, the module sorts the entire edge set in descending order. The system then deterministically deletes the top P_{target} percent of these links, strictly preserving the original node set.

To ensure statistical significance and prevent biased results from a single random sampling, the perturbation module is designed to operate iteratively. For any given degradation threshold (whether utilizing random removals or targeted attacks), the module repeats the masking process for N independent iterations. Instead of outputting a single modified graph, this mechanism generates a batch of perturbed adjacency matrices $(A'^{(1)}, A'^{(2)}, \dots, A'^{(N)})$. This batch is subsequently passed to the generative recovery module, ensuring that the final evaluation reflects true geometric robustness rather than stochastic noise.

3.3. Generative Recovery Module

The Generative Recovery Module processes the Perturbed Graph, denoted as $\{X, A'^{(n)}\}$, through a strict sequence of operations using the pre-trained HypDiff model. To explicitly define the reconstruction process, the module operates through a strict pipeline of three distinct algorithms to probabilistically restore the missing edges, sampling K independent reconstructions for every perturbed graph in the batch:

1. Hyperbolic Geometric Encoding Algorithm

The module processes the perturbed adjacency matrix A' together with the preserved node feature matrix X through a hyperbolic geometric autoencoder based on the Hyperbolic Graph Convolutional Neural Networks (HGCN). Utilizing logarithmic and exponential maps, the system performs the neural network operations within the Euclidean tangent space. This

encoding phase preserves the inherent geometric properties of the surviving structure and directly outputs the continuous hyperbolic latent space representation (z_x).

2. Partial Forward Diffusion and Anisotropic Reverse Denoising Algorithm

Once the perturbed graph is mapped, the system enforces radial and angular geometric constraints. The model then injects partial forward noise, noising the latents only up to a truncation step $t^* < T$ rather than diffusing to pure isotropic noise at the maximum step T . This truncation is what distinguishes recovery from generation: the encoded structure of the surviving topology remains embedded within the noisy latents and actively guides the reverse trajectory, ensuring the model completes the damaged graph rather than generating an unrelated one. Following this injection, the model executes the anisotropic reverse denoising algorithm. This algorithmic process iteratively cleans the noise along specific directional vectors, reusing the hyperbolic community centers computed from the original graph, to probabilistically reconstruct and restore the missing topological links within the latent space.

3. Fermi-Dirac Decoding Algorithm

In the final step of the recovery module, the system passes the cleaned continuous hyperbolic latent representations through a Fermi-Dirac Decoder. This decoding algorithm directly computes the connection probabilities between nodes according to the Fermi-Dirac distribution defined in Section 2.4.1, yielding a continuous probability $P(A_{ij} = 1) \in (0, 1)$ for every node pair. To recover a discrete topology, these probabilities are binarized against a threshold τ , such that an edge is instantiated in the reconstructed adjacency matrix if and only if $P(A_{ij} = 1) \geq \tau$. The value of τ is calibrated experimentally, as specified in Section 4.3, since it directly governs the density of the reconstructed graph and therefore the sensitivity of the final stability score. Repeated executions of this entire algorithmic pipeline on the exact same perturbed graph yield slightly different structural solutions, the module therefore executes it K times per perturbed graph, producing the reconstructions $\hat{A}_{rec,n}^{(1)}, \dots, \hat{A}_{rec,n}^{(K)}$. Across the full batch, this constitutes a Distribution of Reconstructed Graphs comprising $N \times K$ adjacency matrices, which is passed to the scoring stage.

3.4. Probabilistic Stability Scoring Module

The Probabilistic Stability Scoring Module is the final evaluation stage of the system pipeline. It receives the Distribution of Reconstructed Graphs generated by the recovery module, consisting of K stochastic reconstructions for each of the N perturbed graphs, and compares them directly against the absolute structural ground truth provided by the raw input graph, which reaches this module through the bypass connection shown in the architecture diagram. The module transforms these $N \times K$ reconstructions into a single stability score through the following algorithmic steps:

- **Edge Recovery Tracking**

Step 1: Identifying Target Edges. For each perturbed graph $n \in \{1, \dots, N\}$, the module receives the original adjacency matrix A and the corresponding perturbed adjacency matrix $A^{(n)}$. It computes the difference between these matrices to isolate the exact set of removed edges, denoted $E_{removed}^{(n)}$. This set acts as the target features for the recovery evaluation of that specific

perturbation scenario. For every reconstruction $\hat{A}_{rec,n}^{(k)}$, the system records which of these target edges reappear.

Step 2: Edge-Level Recovery Frequency. For each removed edge $e \in E_{removed}^{(n)}$, the algorithm calculates its probabilistic reappearance across the K reconstructions of that perturbed graph:

$$p_e^{(n)} = \frac{1}{K} \sum_{k=1}^K \mathbb{1}[e \in \hat{A}_{rec,n}^{(k)}],$$

where $\mathbb{1}[\cdot]$ is the indicator function. This formulation mathematically quantifies the exact frequency at which the generative model successfully restored a destroyed connection within a single perturbation scenario.

- **Graph-Level Stability Score**

Step 3: The edge-level recovery probabilities are averaged over the removed-edge set of each perturbed graph, producing an individual stability score per perturbation scenario:

$$S_{graph}^{(n)} = \frac{1}{|E_{removed}^{(n)}|} \sum_{e \in E_{removed}^{(n)}} p_e^{(n)}$$

This intermediate score measures how completely the framework restored the specific damage inflicted during iteration n .

- **Final Aggregation**

Step 4: Finally, the system aggregates the graph-level scores across all N independent perturbation iterations. By explicitly averaging over the entire iterative batch, the evaluation is statistically significant and robust against the bias of any single random sampling. The Final Probabilistic Network Stability Score is defined as:

$$S_{stability} = \frac{1}{N} \sum_{n=1}^N S_{graph}^{(n)}$$

This final score provides a quantified, uncertainty-aware measure of the network topology's resilience. A high score indicates that the underlying hierarchical structure contains sufficient geometric information for the algorithm to confidently restore the missing links across multiple independently corrupted states, implying a highly stable network. Because the pipeline is executed once per damage threshold, the resulting sequence of scores forms the degradation curve analyzed in Chapter 4.

4. Evaluation and Verification Plan

4.1. Success Metrics and Criteria

- **Probabilistic Network Stability Score:** This is the primary metric used to evaluate Logical Metric Degradation and Comparative Advantage. The framework will be considered successful if the $S_{stability}$ score plots a smooth, monotonically decreasing degradation curve as the damage threshold increases across the 10% to 50% spectrum, under both random removals and targeted attacks, and if it demonstrates a clear comparative advantage over

classical deterministic measures such as algebraic connectivity, which are expected to exhibit abrupt and unreliable jumps under equivalent stress. A further criterion of success is that targeted attacks produce a measurably faster degradation than random removals of equal magnitude, confirming that the metric is sensitive to the structural importance of the damaged links rather than to damage volume alone.

- **Graph Topological Metrics:** To ensure Generative Consistency, it is necessary to verify that the generated graphs maintain the macroscopic structural integrity of the original networks. The framework will measure the Maximum Mean Discrepancy (MMD) distance, specifically focusing on Degree, Cluster, and Spectre metrics. A lower MMD score will quantitatively prove that the physical topology of the network was preserved despite the perturbations.
- **Fidelity and Diversity Metrics:** Further validating Generative Consistency, the system will use precision-recall ($F1_{pr}$) and density-coverage ($F1_{dc}$) scores. These metrics are critical for proving that the variance in the generated distribution reflects true structural uncertainty. High F1 scores will serve as empirical proof that the system did not suffer from generative failures like “mode collapse”, which would artificially inflate the stability score with repetitive, identical graphs.

4.2. Evaluation Datasets

To comprehensively evaluate the framework’s structural robustness and generative recovery capabilities across diverse network topologies, the system will be tested on a combination of synthetic and real-world datasets. The datasets were selected such that each one isolates and stress-tests a specific geometric constraint of the anisotropic diffusion process.

4.2.1. Synthetic Datasets

The evaluation will utilize standard graph theoretical models to test the framework on controlled, single-nature topologies:

- **Stochastic Block Model (SBM):** Utilized to evaluate the system’s ability to preserve complex, modular community structures under perturbation. This dataset directly stress-tests the angular similarity constraint, since the correctness of the reconstruction depends on whether the removed intra-community edges are restored within their original blocks.
- **Barabási-Albert (BA):** Utilized to assess performance on scale-free networks, specifically testing the preservation of highly connected topological “hubs” during targeted attacks. This dataset directly stress-tests the radial popularity constraint, since targeted attacks by construction concentrate the damage on precisely those hubs.

4.2.2. Real-World Datasets

To validate the framework’s practical applicability and its ability to handle natural structural anisotropy, experiments will be conducted on diverse real-world networks commonly used as benchmarks for graph generation tasks:

- **Social Networks (IMDB-B):** A social network dataset symbolizing the co-starring dynamics of actors, characterized by dense local cliques. This will be used to test the model’s resilience on graphs with distinct human-centric local clustering.

- **Biological Protein Maps (PROTEINS):** A biological network where nodes represent amino acids and edges denote a proximity of less than 6 Angstroms. This dataset will be utilized to evaluate the framework's stability on spatially dense, non-hierarchical topologies, testing whether the hyperbolic prior remains effective on networks that do not exhibit a pronounced hierarchical structure.

4.3. Planned Experiments and Expected Results

Test Category	Experiment Description	Purpose	Expected Insight
Comparative Stability	Compare the $S_{stability}$ score against deterministic metrics (Algebraic Connectivity, Betweenness Centrality) on networks subjected to targeted attacks.	Demonstrate the comparative advantage of the probabilistic framework.	We expect deterministic metrics to show abrupt, unreliable jumps in value, whereas the $S_{stability}$ distribution will smoothly and accurately reflect the true system resilience.
Robustness Degradation	Systematically stress-test networks by increasing the percentage thresholds for Random Edge Removals (P_{rand}) and Targeted Attacks (P_{target}) (e.g., 10%, 20%, 30%, 40%, 50%)	Validate the core perturbation and scoring modules across a spectrum of damage.	We expect the $S_{stability}$ score to plot a smooth downward degradation curve. We hypothesize that P_{target} (targeted attacks) will cause a faster degradation than P_{rand} (random removals).
Generative Consistency Verification	Apply severe perturbations (e.g., >30% removal), run the Generative Recovery Module, and evaluate the output distribution using MMD and F1 scores.	Verify that the variance in the reconstructed graphs reflects true structural uncertainty rather than random algorithmic noise.	We expect the model to maintain strong MMD and F1 scores under heavy damage. This will prove the system avoids mode collapse, ensuring the calculated $S_{stability}$ score is a reliable and accurate measurement of network resilience.
Hyperparameter Calibration	Perform a Grid Search optimization on a validation subset to determine the optimal values for the Fermi-Dirac Hyperparameters (r, t), the edge binarization threshold τ , the diffusion truncation step t^* , and the sampling parameters N and K .	To mitigate metric sensitivity and establish a stable, unbiased baseline for the $S_{stability}$ score before executing the main degradation experiments.	We expect to identify a specific configuration of hyperparameters that maximizes generative stability and minimizes random scoring fluctuations.

4.4. Formal Software Testing Strategy

Test Type	Target Component	Test Description and Execution	Expected Result
Unit Test	Graph Perturbation Module	Input a standard graph and execute the Random Edge Removal and Targeted Attack methods using fixed parameter thresholds (e.g., $P_{rand} = 20\%$ and $P_{target} = 20\%$)	The module outputs a perturbed adjacency matrix with exactly 20% of the relevant edges removed and the original node set together with the node feature matrix X remains structurally intact.
Unit Test	Hyperbolic Autoencoder	Pass a perturbed graph through the HGCN encoder and Fermi-Dirac decoder without adding diffusion noise.	The model encodes and decodes the graph without dimension mismatch errors. The output adjacency matrix shape exactly matches the input, and the latent node embeddings contain no invalid or NaN values.
Unit Test	Stability Scoring Module	Input a mock "Original Graph", a mock perturbed graph, and a mock set of K reconstructions with known differences	The module correctly identifies the exact set of removed edges ($E_{removed}^{(n)}$) and accurately calculates the edge-level recovery frequency $p_e^{(n)}$ against the hand-calculated value, correctly aggregating it into $S_{graph}^{(n)}$ and the final $S_{stability}$ as continuous floats between 0 and 1.
Integration Test	Pipeline Data Flow	Run a small subset of the PROTEINS dataset through the entire sequence: Perturbation $\rightarrow$ Generative Recovery $\rightarrow$ Scoring, using small values of N and K .	The data structures pass smoothly between modules without shape mismatch errors, the recovery module returns exactly $N \times K$ reconstructions and the pipeline outputs a continuous probabilistic score.
System Test	End-to-End Execution	Execute the full degradation experiment (10% to 50% removal) on the complete IMDB-B dataset, repeating the pipeline once per threshold.	The full execution completes seamlessly without crashes, and the system successfully plots the expected downward degradation curve of the $S_{stability}$ score across thresholds.

5. System Infrastructure and Project Roadmap

5.1. System Requirements

Functional Requirements

- **Graph Ingestion and Preprocessing:** The system shall receive raw complex network datasets as input and convert them into the standard representation $G = \{X, A\}$, consisting of a node feature matrix and a discrete adjacency matrix.
- **Iterative Edge Removal (Perturbation):** The system shall execute a predefined number of perturbation iterations (N) per damage threshold. In each iteration, the system shall actively remove a specific fraction of edges from the network while leaving the node set and the node feature matrix X unchanged, outputting a batch of N perturbed graphs.
- **Hyperbolic Geometric Encoding:** The system shall process the damaged graphs from each iteration and map them into a continuous hyperbolic latent space using HGCN.
- **Generative Edge Reconstruction:** The system shall apply partial forward diffusion up to a truncation step $t^* < T$ followed by an anisotropic reverse diffusion process to the latent representations, attempting to predict and computationally reconstruct the exact topological links that were removed, sampling K independent reconstructions per perturbed graph.
- **Probabilistic Stability Scoring:** The system shall track the recovery frequency of each removed edge across the K reconstructions, aggregate these frequencies into a per-graph score for each of the N iterations, and output a final numerical stability score ($S_{stability}$) as the mean of the per-graph scores.
- **Degradation Sweep:** The system shall repeat the complete pipeline once per damage threshold across a configurable range (e.g., 10% to 50%) and output the resulting degradation curve.

Non-Functional Requirements

- **Scalability:** The system must efficiently handle complex scale-free networks by embedding them into hyperbolic space, actively avoiding the geometric distortion caused by Euclidean spaces.
- **Robustness:** The anisotropic diffusion process must maintain structural stability and prevent the signal-to-noise ratio from rapidly decaying under the injected topological noise.
- **Computational Efficiency:** The system must perform standard statistical operations locally within the Euclidean tangent space utilizing logarithmic and exponential maps.
- **Reproducibility:** The system must log all perturbation parameters, the sampling parameters N and K , and the truncation step t^* and utilize fixed random seeds to ensure that repeated executions yield consistent and reliable stability scores.
- **Modularity:** The software pipeline must be modularly structured, ensuring that the perturbation module and the scoring module can be updated, expanded, or replaced without requiring modifications to the core generative diffusion engine.

5.2. Expected Challenges

- **Architectural Adaptation:** Adapting the generative HypDiff framework to measure stability requires balancing sampling stability to reconstruct removed edges without overwriting the remaining structure. Selecting the truncation step t^* is central to this balance, since too large a value discards the surviving topology while too small a value prevents the model from generating meaningful structural variation.
- **Geometric Instability in Hyperbolic Embeddings:** Targeted or random edge disruptions can drastically warp the network's underlying hierarchical geometry, causing unpredictable coordinate shifts during HGCN encoding.
- **Calibration and Sensitivity of the Network Resilience Score:** The newly proposed $S_{stability}$ metric is highly sensitive to baseline graph density and Fermi-Dirac hyperparameters, which may yield unstable or skewed resilience scores.
- **Computational and Run-Time Overhead:** The combined use of hyperbolic K-Means clustering, repeated tangent plane projections and running K reverse denoising iterations for each of the N perturbed graphs ($N \times K$ total reconstructions per damage threshold, repeated across the full threshold sweep) substantially increases computational complexity.
- **Controlling Generative Hallucinations:** The decoder may generate topologically plausible but historically non-existent links, creating false positives that could artificially inflate the final network stability score.

5.3. Future Work Plan and Project Roadmap

The project lifecycle is structured into distinct stages, progressing from theoretical mathematical foundations to practical software verification. Following the geometric and architectural design presented in Phase A, the roadmap for Phase B focuses on the translation of these continuous-time diffusion concepts into a fully operational and deployable evaluation framework. The work plan consists of the following primary development stages:

1. **Theoretical Design:** Formulation of the mathematical methodology for the stability framework and completion of the formal system architecture and testing plan.
2. **Pipeline Implementation:** Development of the Python/PyTorch codebase, focusing on building the Graph Perturbation Module, integrating the pre-trained HypDiff autoencoder, and implementing the Probabilistic Stability Scoring module.
3. **Experimental Validation:** Execution of the formal testing strategy on real-world datasets to empirically calibrate noise, calculate the $S_{stability}$ metric, and plot degradation curves.
4. **Analysis and Final Documentation:** Interpretation of the empirical results, including the behavior of the degradation curves and their comparison against the classical deterministic metrics, consolidation of the findings, and documentation of the conclusions in the final project report.

Beyond the scope of the development stages outlined above, the framework may be further benchmarked against alternative graph generation baselines, such as the Euclidean diffusion models GDSS and DiGress, under identical perturbation thresholds. Such a comparison would

isolate and empirically quantify the specific contribution of hyperbolic geometry to structural recovery, and it is therefore identified as the primary direction for future work.

5.4. Ai Tools Usage

Phase	Tool	Prompt Used	Purpose
Literature Review	NotebookLM	“Summarize the mathematical formulations and geometric constraints of the HypDiff latent diffusion model from the uploaded source.”	Analyzing complex mathematical research papers and extracting the core non-Euclidean graph embedding formulas.
Mathematical Background	Perplexity	“How do hyperbolic graph neural networks utilize the Fermi-Dirac decoder to calculate connection probabilities?”	Searching the web for real-time academic explanations and cross-referencing geometric machine learning concepts.
System Design	Gemini	“What are the expected algorithmic challenges when adapting a generative HypDiff model to perform guided edge reconstruction instead of pure generation?”	Brainstorming and identifying practical software engineering bottlenecks for Phase B.
Algorithmic Logic	Gemini	“Explain the geometric instability in hyperbolic embeddings when targeted edge disruptions occur.”	Refining the definitions of mathematical risks associated with the continuous-time pipeline.
Writing and Editing	Gemini/ NotebookLM	“Refine this text for academic tone: ‘The project uses a generative model to find the true stability of the network.’”	Improving the clarity and professional tone of the documentation.

References

- Artime, O., Grassia, M., De Domenico, M., Gleeson, J. P., Makse, H. A., Mangioni, G., ... & Radicchi, F. (2024). Robustness and resilience of complex networks. *Nature Reviews Physics*, 6(2), 114-131.
- Chami, I., Ying, Z., Ré, C., & Leskovec, J. (2019). Hyperbolic graph convolutional neural networks. *Advances in neural information processing systems*, 32.
- Elhag, A. A., Corso, G., Stärk, H., & Bronstein, M. M. (2022, April). Graph anisotropic diffusion for molecules. In *ICLR2022 Machine Learning for Drug Discovery*
- Ellens, W., & Kooij, R. E. (2013). Graph measures and network robustness. *arXiv preprint arXiv:1311.5064*.
- Fu, X., Gao, Y., Wei, Y., Sun, Q., Peng, H., Li, J., & Li, X. (2024). Hyperbolic geometric latent diffusion model for graph generation. *arXiv preprint arXiv:2405.03188*.
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33, 6840-6851.
- Huang, H., Sun, L., Du, B., Fu, Y., & Lv, W. (2022, November). Graphgdp: Generative diffusion processes for permutation invariant graph generation. In *2022 IEEE International Conference on Data Mining (ICDM)* (pp. 201-210). IEEE.
- Jo, J., Lee, S., & Hwang, S. J. (2022, June). Score-based generative modeling of graphs via the system of stochastic differential equations. In *International conference on machine learning* (pp. 10362-10383). PMLR.
- Nickel, M., & Kiela, D. (2017). Poincaré embeddings for learning hierarchical representations. *Advances in neural information processing systems*, 30.
- Papadopoulos, F., Kitsak, M., Serrano, M. Á., Boguná, M., & Krioukov, D. (2012). Popularity versus similarity in growing networks. *Nature*, 489(7417), 537-540.
- Vignac, C., Krawczuk, I., Siraudin, A., Wang, B., Cevher, V., & Frossard, P. (2022). Digress: Discrete denoising diffusion for graph generation. *arXiv preprint arXiv:2209.14734*.
- Yang, R., Yang, Y., Zhou, F., & Sun, Q. (2023). Directional diffusion models for graph representation learning. *Advances in neural information processing systems*, 36, 32720-32731.